eb, workhorse, sidekiq, and gitaly indices.

Rate Limit & Requests Dashboard

Use this dashboard to quicky filter records by username, IP, path, or namespace. This is useful when you are trying to determine if a User or IP is being subject to rate limiting and you need to discover and visualize usage patterns.

Integrations Dashboard

This can be useful to track down errors related to specific integrations across an entire namespace.

Errors & Exceptions Dashboard

This dashboard can be used to get specific http statuses across a namespace or path/project and narrow down by class. This can be useful for understanding error patterns across a namespace or even across the entirety of GitLab.com.

Tips and Tricks

This section details how you can find very specific pieces of information in Kibana by searching for and filtering out specific fields. Each tip includes a link to the group, subgroup, or project that was used in the example for reference.

Runner Registration Token Reset

Example group: gitlab-bronze

We can determine if the GitLab Runner registration token was reset for a group or project and see which user reset it and when. For this example the Runner registration token was reset at the group-level in gitlab-bronze. To find the log entry:

1. Set the date range to a value that you believe will contain the result. Set it to Last 7 days if you're unsure.
1. Add a positive filter on json.path for the path of the group, which is just gitlab-bronze in this example.
1. Add a positive filter on json.action for resetregistrationtoken.
1. Observe the results. If there were any they will contain the username of the user that triggered the reset in the json.username field of the result.

Access Token activity

We can determine the kind of activities an Access Token (Group, Project, Personal) is performing. To find the log entry:

1. Find the id of the Access Token you are interested in using the API or UI.
1. In pubsub-rails-inf-gprd-, set the date range to a value that you believe will contain the result. Set it to Last 7 days if you're unsure.
1. Add a positive filter on json.tokenid for the id in step 1.

1. Add other filters that you might be interested in:
   - json.username
   - json.path
   - json.method
   - json.tokentype

Deleted Group/Subgroup/Project

- Example group: gitlab-silver
- Example project: gitlab-silver/test-project-to-delete

Kibana can be used to determine who triggered the deletion of a group, subgroup, or project on GitLab.com. To find the log entry:

1. In pubsub-rails-inf-gprd-, set the date range to a value that you believe will contain the result. Set it to Last 7 days if you're unsure.
1. Add a positive filter on json.path for the path of the project, including the group and subgroup, if applicable. This is gitlab-silver/test-project-to-delete in this example.
1. Add a positive filter on json.method for DELETE.
1. Observe the results. If there were any they will contain the username of the user that triggered the deletion in the json.username field of the result.
When a project or a group is first going to pending deletion the log entry will have json.params.key: method, authenticitytoken, namespaceid, id], compare to when a user is [forcing the deletion then the log entry looks like json.params.key: [method, authenticitytoken, permanentlydelete, namespaceid, id] for project or looks like json.params.key: [method, authenticitytoken, permanentlyremove, id] for group.

To see a list of projects deleted as part of a (sub)group deletion, in sidekiq:

1. Filter json.message to "was deleted".
1. Set a second filter json.message to path/group.

Viewed CI/CD Variables

While we do not specifically log changes made to CI/CD variables in our audit logs for group events, there is a way to use Kibana to see who may have viewed the variables page. Viewing the variables page is required to change the variables in question. While this does not necessarily indicate someone who has viewed the page in question has made changes to the variables, it should help to narrow down the list of potential users who could have done so. (If you'd like us to log these changes, we have an issue open here to collect your comments.)

1. Set a filter for json.path is and then enter the full path of the associated project in question, followed by /-/variables. For example, if I had a project named tanuki-rules, I would enter tanuki-rules/-/variables.
1. Set the date in Kibana to the range in which you believe a change was made.
1. You should be able to then view anyone accessing that page within that time frame. The json.meta.user should show the user's username and the json.time should show the timestamp during which the page was accessed.
1. You may also be able to search by json.graphql.operationname is getProjectVariables.
1. In general, a result in the json.action field that returns update and json.controller

returning Projects::VariablesController is likely to mean that an update was done to the variables.

Find Problems with Let's Encrypt Certificates

In some cases a Let's Encrypt Certificate will fail to be issued for one or more reasons. To determine the exact reason, we can look this up in Kibana.

1. Set a positive filter for json.message to "Failed to obtain Let's Encrypt certificate".
1. In the search bar, enter in the GitLab Pages domain name. For example json.pagesdomain: "sll-error.shushlin.dev"
1. Filter by or find json.acmeerror.detail This should display the relevant error message. In this case, we can see the error "No valid IP addresses found for sll-error.shushlin.dev"

Deleted User

Kibana can be used to find out if and when an account on GitLab.com was deleted if it occurred within the last seven days at the time of searching.

- Set the date range to a value that you believe will contain the result. Set it to Last 7 days if you're unsure.

If an account was self-deleted, try searching with these filters:

1. Add a positive filter on json.username for the username of the user, if you have it.
1. Add a positive filter on json.controller for RegistrationsController.

If an account was deleted by an admin, try searching with these filters:

1. Add a positive filter on json.params.value for the username of the user.
1. Add a positive filter on json.method for DELETE.

Observe the results. There should be only one result if the account that was filtered for was deleted within the specified timeframe.

If you suspect an account was deleted by the cron job that deletes unconfirmed accounts, try searching with these filters:

1. Change to the pubsub-sidekiq-inf-gprd index.
1. Add a positive filter on json.meta.user for the username of the user. (Alternatively, you can use json.args.keyword and use the User ID of the user if you have that).
1. Add a positive filter on json.class for DeleteUserWorker.

Disable Two Factor Authentication

- Quick link to search

Kibana can be used to find out which admin disabled 2FA on a GitLab.com account. To see the log entries:

1. Set the date range to a value that you believe will contain the result. Set it to Last 7 days if you're unsure.
1. Add a positive filter on json.location for the username 2FA was disabled for.
1. Add a positive filter on json.action for disabletwofactor.
1. To make things easier to read, only display the json.location and json.username fields.
1. Observe the results.

Enable/Disable SSO Enforcement

Kibana can be used to find out if and when SSO Enforcement was enabled or disabled on a group on GitLab.com, and which user did so.

- Example group: gitlab-silver

Enable SSO Enforcement

1. Set the date range to a value that you believe will contain the result. Set it to Last 7 days if you're unsure.
1. Add a positive filter on json.controller for Groups::SamlProvidersController.
1. Add a positive filter on json.action for update.
1. Add a positive filter on json.method for PATCH.
1. Add a positive filter on json.path for the path of the group, gitlab-silver in this example case.
1. If there are any results, observe the json.params field. If \"enforcedsso\"=>\"1\" is present, that entry was logged when SSO Enforcement was enabled by the user in the json.username field.

Disable SSO Enforcement

1. Set the date range to a value that you believe will contain the result. Set it to Last 7 days if you're unsure.
1. Add a positive filter on json.controller for Groups::SamlProvidersController.
1. Add a positive filter on json.action for update.
1. Add a positive filter on json.method for PATCH.
1. Add a positive filter on json.path for the path of the group, gitlab-silver in this example case.
1. If there are any results, observe the json.params field. If \"enforcedsso\"=>\"0\" is present, that entry was logged when SSO Enforcement was disabled by the user in the json.username field.

SAML log in

To investigate SAML login problems:

In the pubsub-rails-inf-gprd- log:

1. Set the date range to a value that you believe will contain the result. Set it to Last 7 days if you're unsure.
1. Add a positive filter as advised in our SAML groups docs.

After decoding the SAML response, and observing the results corresponding to your chosen filters, you can see if there are any missing or misconfigured attributes.

SCIM provisioning and de-provisioning

To investigate SCIM problems:

In the pubsub-rails-inf-gprd- log:

1. Set the date range to a value that you believe will contain the result. Set it to Last 7 days if you're unsure.
1. Add a positive filter on json.path for:
   - /api/scim/v2/groups/<group name> when looking at the SCIM requests for a whole group. This path can also be found in the group's SAML settings.
   - /api/scim/v2/groups/<group name>/Users/<user's SCIM identifier> when looking at the SCIM requests for a particular user.
1. Add a positive filter on json.methhod for POST or PATCH (first time provisioning or update/de-provisioning respectivley).
1. Add a filter on json.params.value to further identify specific actions:
   - json.params.value = Add reveals first time provisioning, or account detail changes
   - json.params.value = Replace reveals SCIM deprovisioning, happens when a user has been removed from the IdP App/group
   - json.params.value = <External ID> reveals any SCIM activity related to the identified External ID.

In cases where the SCIM provisioned account is deleted:

1. Follow the above steps and get the correlationid from the provisioning record with POST as json.method.
1. Go to the Correlation Dashboard and search for the relevant correlationid.
1. In the results, look for records in the Correlation Dashboard - Web section, and an entry with elasticsearch as json.subcomponent. In that entry, find the userid in json.trackeditemsencoded in the format of [[numbers,"User <userid> user<userid>"]].

To investigate if the user was deleted due to an unconfirmed email, follow the Deleted User procedure.

Searching for Remove User from group or subgroup

If it happened within the retention period (7 days), Kibana can be used to determine if, when and by whom a user was removed from a group or subgroup

To find the log entry in pubsub-rails-inf-gprd- with the following data points:

Confirm the Remove User (DELETE)

1. Add a positive filter on json.meta.callerid for Groups::GroupMembersControllerdestroy
1. Add a positive filter on json.meta.userid for user id of person that performed the remove user action in the UI
1. Add a positive filter on json.method for DELETE

Retrieve further details about the Remove User request

The following filters can help identify users that were removed and what group or subgroups they have been removed from

1. Add a positive filter on json.custommessage for Membership destroyed
1. Add a positive filter on json.meta.callerid for Groups::GroupMembersControllerdestroy
1. Add a filter for user id json.meta.userid or username json.meta.user of the user that performed the Remove User action
1. Add a filter for target user id json.details.targetid

Searching for Deleted Container Registry tags

Kibana can be used to determine whether a container registry tag was deleted, when, and who triggered it, if the deletion happened in the last 7 days.

To find the log entry in pubsub-rails-inf-gprd-:

1. Get the link for the container registry in question. (For exmaple: <https://gitlab.example.com/group/project/containerregistry/><ContainerRepository>)
1. Set the date range to a value that you believe will contain the result. Set it to Last 7 days if you're unsure.
1. Add a positive filter on json.graphql.variables for ContainerRepository/<regitsry id>..
1. Add a positive filter on json.graphql.operationname for destroyContainerRepositoryTags.

Searching Kibana for 500 level errors

As of 14.7, a Correlation ID is provided on the 500 error page when using the interface. You can ask the customer to supply this and use Kibana to filter by json.correlationid.

Kibana is not typically used to locate 5XX errors, but there are times where they can't be easily found in Sentry and searching Kibana first is beneficial. To perform a general search in Kibana:

1. Obtain the full URL the user was visiting when the error occurred.
1. Log in to Kibana.
1. Select the correct time filter (top right) - e.g last 30 minutes, 24 hours, or 7 days.
1. In the search field, type json.path : "thepathaftergitlab.comfromtheURL"
1. Choose relevant fields from the sidebar. For a 500 error, you want to filter for json.status and choose is, then enter 500.
1. Continue to use relevant fields from the list on the sidebar to narrow down the search.

See the 500 errors workflow for more information on searching and finding errors on GitLab.com

Filter by IP Range

Customers will sometimes give us an IP Range of their resources such as their Kubernetes cluster or other external servers that may need to access GitLab. You can search a range by using Elasticsearch Query DSL.

1. Click + Add Filter > Edit as Query DSL
1. Use the following format to create your query:

```
json
{
  "query": {
    "term": {
      "json.remoteip": {
        "value": "192.168.0.1/20"
      }
    }
  }
}
```

1. Click Save to perform your query.

Note that depending on the range, this operation may be expensive so it is best to first narrow down your date range first.

Import Errors

Most timeout related imports end up with a partial import with very few or zero issues or merge requests. Where there is a relatively smaller difference (10% or less), then there are most likely errors with those specific issues or merge requests.

Anytime there is an error, ensure that the export originated from a compatible version of GitLab.

Here are some tips for searching for import errors in Kibana:

- Use the pubsub-sidekiq-inf-gprd index pattern (Sidekiq logs) and try to narrow it down by adding filters
  - json.meta.project: path/to/project
  - json.severity: (not INFO)
  - json.jobstatus: (not done)
  - json.class is RepositoryImportWorker
- Use the pubsub-rails-inf-gprd index pattern (Rails logs) and try to narrow it down by adding filters
  - json.controller: Projects::ImportsController with error status
  - json.path: path/to/project
- In Sentry, search/look for: Projects::ImportService::Error ; make sure to remove the is:unresolved filter.

If there is an error, search for an existing issue. Errors where the metadata is throwing an error and no issue exists, consider creating one from Sentry.

If no error is found and the import is partial, most likely it is a timeout issue.

Export Errors

Export errors can occur when a user attempts to export via the UI or this API endpoint. A parameter in the API allows for exporting to an external URL such as a pre-signed AWS S3 URL. Typically, the export process consists of:

- Returning an initial 202 response to the client confirming the export has started
- Taking from a few seconds to a few minutes to process the export, depending on the project size
- Starting the upload to the specified upload URL

Here are some suggestions for searching export logs in Kibana:

- In pubsub-sidekiq-inf-gprd (Sidekiq), narrow the search by adding filters:
  - json.class: Projects::ImportExport
  - json.meta.project path/to/project

The Projects::ImportExport class will include the following subcomponents:

- Projects::ImportExport::RelationExportWorker
- Projects::ImportExport::CreateRelationExportsWorker
- Projects::ImportExport::ParallelProjectExportWorker
- Projects::ImportExport::WaitRelationExportsWorker
- Projects::ImportExport::AfterImportMergeRequestsWorker

If using the Correlation Dashboard, you should be able to follow Sidekiq events throughout the export process, and locate errors by filtering json.severity: ERROR. Provided below is an example of an upload failing to AWS S3:

json
"severity": "ERROR",
    "meta.callerid": "ProjectExportWorker",
    "subcomponent": "exporter",
    ...
    "message": "Invalid response code while uploading file. Code: 400",
    "responsebody": "<?xml version=\"1.0\"
encoding=\"UTF-8\"?>\n<Error><Code>EntityTooLarge</Code><Message>Your proposed upload exceeds
the maximum allowed size</Message><ProposedSize>6353524398</ProposedSize><MaxSizeAllowed>53
9120</MaxSizeAllowed><RequestId><omitted></RequestId><HostId><omitted></HostId></Error>",

The XML response can provide an indication of the failure reason, such as the project exceeding the maximum allowed size:

xml
<?xml version="1.0" encoding="UTF-8"?>
<Error>
  <Code>EntityTooLarge</Code>
  <Message>Your proposed upload exceeds the maximum allowed size</Message>
  <ProposedSize>6353524398</ProposedSize>
  <MaxSizeAllowed>5368709120</MaxSizeAllowed>
  <RequestId>

<omitted>
</RequestId>
<HostId>
<omitted>
</HostId>
</Error>

Known Import Issues

- Imported project's size differs from where it originated
  - See this comment for an explanation as to why. As artifacts are part of repository size, whether they are present can make a big difference.
- Repository shows 0 commits.
  - See 15348.

Purchase Errors

Kibana can be used to search for specific errors related to a purchase attempt. Since purchases can be made either in GitLab.com or CustomersDot, it is important to determine which system a customer was using to make the purchase; then use the tips specific to the system below. Note: the ticket submitter might not be the customer making the purchase.

GitLab.com purchase errors

Note: You need to have the GitLab username of the account used to make the purchase. Sometimes the user fills the GitLab username value of the ticket fields, or you can check the ticket requester's GitLab username in the User Lookup in the GitLab Super App.

1. Navigate to Kibana
1. Ensure the pubsub-rails-inf-gprd- index pattern (GitLab.com logs) is selected.
1. Set the date range to a value that you believe will contain the result. Set it to Last 7 days if you're unsure.
1. Add a positive filter on json.user.username for the username of the account used to make the purchase.
1. Add a positive filter on json.tags.featurecategory for purchase.

Feeling Lazy? Go to https://log.gprd.gitlab.net/goto/6aac4580-9d9b-11ed-85ed-e7557b0a598c and update the value of json.user.username.

If you encounter a generic error message try checking CustomersDot purchase error logs in Kibana or GCP for a more specific error.

Tip: To see the details of a user's purchase attempt, go to https://log.gprd.gitlab.net/goto/45bb89c0-6ccc-11ed-9f43-e3784d7fe3ca and update the value of json.username.

CustomersDot purchase errors

Note: You need to have the CustomersDot customer ID of the account used to make the purchase.

Refer to Step 1 under this section on how to get the customer ID.

1. Navigate to Kibana
1. Ensure the pubsub-rails-inf-prdsub- index pattern (CustomersDot logs) is selected.
1. Set the date range to a value that you believe will contain the result. Set it to Last 7 days if you're unsure.
1. Add a positive filter on json.customerid for the username of the account used to make the purchase.
1. Add a positive filter on json.severity for ERROR.

Feeling Lazy? Go to https://log.gprd.gitlab.net/goto/9c28ba80-9d9b-11ed-9f43-e3784d7fe3ca and update the value of json.customerid.

In case you have the namespace details, get the Namespace ID then go to https://log.gprd.gitlab.net/goto/b598dfe0-9d9b-11ed-9f43-e3784d7fe3ca and update the value of json.params.glnamespaceid

Check user actions on a repository

When looking at the pubsub-rails-inf-gprd- index, you can determine if a user has recently cloned/pushed (with HTTPS), or downloaded a repository. You can filter by json.username, json.path (the repository), and json.action to find specific events:

- action: gituploadpack is when someone performs a clone of a repository.
- action: gitreceivepack is when someone push's a repository.
- action: archive is when someone downloads a repository via the Download source code button in the UI.

To search for git activity over SSH, you can instead look in the pubsub-shell-inf-gprd- index for specific events (note the minus signs instead of underscores):

- command: git-upload-pack is when someone performs a clone over SSH.
- command: git-receive-pack is when someone performs a push over SSH.

You can use the links in the lists above and fill in the json.path or json.glprojectpath for the project of interest.

Webhook related events

Webhook events for GitLab.com can be located in Kibana, including identifying when a group or project has gone over enforced rate limits. Rate limiting varies depending on the subscription plan and number of seats in the subscription.

Here are some suggestions:

- Use pubsub-sidekiq-inf-gprd (Sidekiq) with the filters json.class: "WebHookWorker" and json.meta.project : "path/to/project" to identify webhook events.
- Use pubsub-rails-inf-gprd- (Rails) with the filters json.message : "Webhook rate limit exceeded" and json.meta.project : "path/to/project" to identify webhooks that failed to send due to rate limiting.

- You can filter between Group and Project hooks by using json.meta.relatedclass : "GroupHook" or json.meta.relatedclass : "ProjectHook".

Searching for Service Desk emails

When searching through Kibana for the json.toaddress, make sure this is the address that appears on the to: line in the email, even if this is aliased to the GitLab project email address. If you search for the project email address and the Service Desk mail was sent to an alias of that (support@domain.ext for example), it won't show up in the searches.

SECTION: support/workflows/knowledge_areas.md

Overview

The following internal pages provide a reference of Support team members who feel that they are knowledgeable and willing to assist with tickets in a particular area.

- Areas of Focus
- Skills by Person
- Skills by Subject

Unlike the GitLab Team page, team members listed here may or may not be an expert as defined in the Handbook. For example, they may be working on a module but have not completed it. Additionally, some areas may not currently have a module, or the knowledge area is very specific, such as "GitLab.com SSO".

When to Use

At GitLab, we avoid Direct Messages in Slack. Therefore, use discretion when trying to locate a team member with specific knowledge. It is almost always better to send a message to one of the Support Slack channels, the relevant Stage/Group channel, a technology channel (for example, kubernetes), or questions. You can also check the GitLab team page for non-support team members who can help.

NOTE: When deciding whether to ping a team member, first consult the internal Areas of Focus page, because team members allocate their time and expertise differently.

How to update your knowledge areas

Knowledge areas, skills, and Areas of Focus are maintained in the your support-team project entry page.

Consult the support-team project README for instructions on updating your entries.

SECTION: support/workflows/license_subscription_workflows.md

The license and subscription workflows handled by L&R-focused Support Engineers have been moved to a dedicated L&R workflows page.

SECTION: support/workflows/log_requests.md

Log requests for GitLab.com

Users often ask for access to GitLab.com logs, typically, due to IP blocks, a possible security issue, or for internal auditing purposes.

Always include a link to the log as an internal note, with additional information if needed.

A standard response is available in ZenDesk as a macro Support::SaaS::Gitlab.com::Audit logs access request.

If required, you can escalate the ticket/issue by following our escalation process.

You can consider using the kibana workflow page for tips on retrieving logs for requests within the last 7 days. Log requests beyond a summary (similar to the examples below) or where logs are not readily available on Kibana should be handled according to the process outlined in the handbook page dedicated to providing assistance to GitLab.com customers during customer-based security incidents. GitLab's Security Incident Response Team handles complex, extensive requests according to an internal runbook for customer response operations.

Who can make a request

Paid Subscriptions

Requester must be a Group Owner of a pre-existing paid namespace.

- Must verify that this is who is making the request and should be in alignment with support for Enterprise Users

> NOTE: A user cannot upgrade to a paid subscription to gain access to logging requests.

Free Users

Free users should reference GitLab.com rate limits documentation. Support will provide information when GitLab initiates contact due to an incident.

What we can provide

We can provide the following information:

- Information found in the Audit Events Features
- Information about who has accessed the account/projects that the customers owns. This can include:
  - number of users

  - number of times accessed
  - number of unique IPs
  - range of timestamp of occurrence
  - project path
- Provide the above excluding a known list. For example, number of IP addresses not originating from a user's "work office".

What we cannot provide

We cannot provide the following information:

- Information about accounts or projects that the requester does not own.
- Any information considered Personal Data that is not specifically about the individual requester.
- Any information that would disclose GitLab confidential information or processes.

Sending logs and other Personal Data

Any Personal Data information that is pulled by the Security Incident Response Team (SIRT), such as a log request, needs to be delivered compressed and password protected to the requestor with the following guidelines:

- The password should be a random string of at least 10+ characters including numbers, lower and upper case letters.
- The password protected file should be attached to the ZenDesk ticket, and the password needs to be sent separately through your email account directly to the customer's email address.
  - Use the command zip -er [TicketNumber].zip filename or other encryption tool to encrypt the file.
  - Use 1Password to generate the random secure password for the encryption.
- Once the customer had successfully received and opened the files you should delete the pulled data from your computer and the email from your mailbox.

If the log files are too large to attach to the ticket in ZenDesk, refer to the Provide large files to GitLab support page.

In case you need to share the data pull results internally, such as in an internal issue, upload the files to Google Drive, such as the Support Ticket Attachments folder (internal).

Examples

The following are examples to provide a better idea of what responses we can provide.

Example 1: Who accessed a specific repo

A customer, who had accidentally set their project to the incorrect visibility setting, wanted to know if anyone outside the company accessed their project. Here is a modified excerpt of the response:

> Excluding users who have the company email domain, 2 users viewed the main project page a total of 4 times between 20:06 and 20:10 UTC 2019-08-15. However, I can confirm that all 4

instances originated from one of the IP addresses you provided as being from your office.

From ticket: 129594

Example 2: IP block

User writes in to say their entire team is getting blocked and they want to know the source. When the user writing in has access to the projects in question, we can provide the specific path(s).

> It appears that the majority of requests that returned 401, which likely caused the temporary block, involved /project/path.

Example ticket: 132652

Also see "Identifying the Cause of IP Blocks on GitLab.com".

Example 3: GitLab requests action due to high load

GitLab reached out to the owners of a project that was causing concern for the production team, who asked Support to reach out. The user wanted to know where the requests were originating from.

> There are 3 different IPs showing in our logs, 2 of which are based in CountryA and 1 in CountryB (please note these locations may not be accurate as they are based purely on geolocation web searches). They also all have the same user agent.

Example ticket: 130153

SECTION: support/workflows/looking_up_customer_account_details.md

Looking up customer account details

While working on tickets, you may need to look up customer information. Common use cases include associating tickets with the appropriate organization, checking a customer's subscription plan and looking up the customer's technical account manager.

In general, you should look for customer details in this order:

1. GitLab User Lookup app in Zendesk
1. Within customers.gitlab.com
1. Within licenses in CustomersDot
1. Within Salesforce (if you have access)

For an overview and runthrough of manual searching of SFDC, customers.gitlab.com, watch Amanda Rueda's
How to use Salesforce from a support perspective

video.

GitLab User Lookup app in Zendesk

From the Zendesk GitLab User Lookup application you have access to the requester details in SFDC and GitLab.com.

Within customers.gitlab.com

1. Log in to customers.gitlab.com admin area
   (sign in with Okta).

1. In the Customers section, search for a domain or full email address.

1. In the search results, click on the i icon to view the customer's details.

1. You can impersonate an account to find out if they have a current
   subscription through the customer's detail page or by clicking on the home
   icon in the search results.

Note: be extra careful when searching using the customer's domain: there can be generic domains that you are not aware of, and there can be large customers with multiple organizations using the same
domain. Therefore, search by e-mail is more reliable.

Within licenses in customers.gitlab.com

All self-managed licenses including trial ones should be available in CustomerDot Licenses. Access is provisioned via Okta.

When the license ID is provided

If a customer provides you with their license ID, you may need to check for it in CustomersDot.

You can do so by appending the ID to the following URLs:

- <https://customers.gitlab.com/admin/license/>

  e.g. https://customers.gitlab.com/admin/license/<LICENSEID>

When the full license file is provided

Sometimes a customer may include the full license file to prove their support entitlement. There are two methods to decode a license. One method is to use a script and the other is to use the Rails console on a self-managed instance.

License Decoder

The easiest method is to use the License Decoder ruby script.
It outputs nice clean information including links to subscription information
and a direct link to the CustomersDot License page.

Follow the instructions in the project
for installation and usage instructions.

From the Rails console

You can determine the license ID (and thus organization) by
extracting the ID.

First, trim the carriage returns and/or new lines:

```
text
tr -d '\r\n' < filename.gitlab-license
```

Then, from the Rails console on your own self-managed instance:

```
text
license = ::License.new(data: "<paste entire license key without the carriage returns>")
"https://customers.gitlab.com/admin/license/".concat(license.licenseid.tos)
```

This will return nice URL that will take you the relevant license in CustomersDot.

```
text
=> "https://customers.gitlab.com/admin/license/<licenseid>"
```

Within Salesforce

If you have access, you have the ability to look up the ticket requester's organization
directly in Salesforce.

Finding the customer's organization

1. Search for the customer's domain (e.g. customer.com) or full email address
   (e.g. flastname@customer.com) in the search bar at the top of the
   Salesforce UI.

1. Look for results in the Accounts section. You should also be able to see
   if they have a support level if they have one.

1. Click the Account Name to view the customer's organization page.

Note: in some cases you will need to search by e-mail and by domain. For example, if the e-mail has previously been associated with a trial account it will still be visible in SFDC but this might not be the same account that is used by the organization.

Finding the customer's GitLab subscription information

In the customer's organization page, look for the GitLab Subscription Information section. The most relevant pieces of information to support in this section are:

- Support Level: Whether they are on starter or premium tier support
- GitLab Plan (TEST): Which subscription plan they are on
- Number of Licenses: The number of license seats which the customer is paying for
- CARR: The total annual recurring revenue this customer brings

You can also confirm if the organization is a paying customer by look for the Type field under the Account Detail section. It should say Customer.

Finding the customer's account owner

In the customer's organization page, look for the Account Detail section. There should be an Account Owner field. This is the person responsible for the customer account.

Alternatively, look for the list of links just above the Account Detail section. Note: You may have to wait awhile as the list only loads after the rest of the page is loaded.

Hover over the "Account Team" link to see a list of people who have handled the customer account.

Finding the customer's renewal opportunity owner

In the customer's organization page, look for the Opportunities table. Look for a row with a Close Date in the future and a stage that is not Closed Won or 10-Duplicate. This should generally be the first row.

The person responsible for the customer's license renewal is listed under Owner Full Name.

SECTION: support/workflows/looking_up_customer_technical_details.md

While troubleshooting customer issues in tickets, you may find yourself needing additional context -- a gitlab.rb file, an architectural diagram, or anything which will help you move forward with your investigation.

There are a few places where you can find this information.

Within Zendesk

Internal comment with Organization Notes

When a new ticket comes in and there is an organization attach to this ticket, there will be a Zendesk automation trigger (Ticket::Internal Comment::Organization Info) that puts an internal comment to the ticket. This internal comment will include organization notes if it exists. These organization notes are saved within Zendesk, visible to agents only, not to end-users.

During your work on the ticket, if you have additional information worth noting about the organization, you can add them by following the editing organizations.

You may also consider updating the Customer Collaboration Projects within GitLab.com describe below.

Browse previous tickets

You can browse other tickets submitted by people from the customer's organization for relevant context! Often forgotten in the heat of the moment.

Just click on the organization name at the top of the ticket to view all other tickets associated with that organization.

Alternatively, you can do a search for organization:<$ORGNAME> search query, which may get you the information you want. Try "gitlab.rb" or gitlabsos for your search!

In both cases, you can click on Requested or Updated to sort by most recent so that you'll be sure to have fresher information.

Architecture diagram and Customer Collaboration Project

The Architecture Diagrams app automatically checks for the presence of the relevant diagram if the customer has a Customer Collaboration Project URL entered in Salesforce.

To access the app:

1. Click on "Apps" in the top right of the Zendesk UI

1. Look for the Architecture Diagrams app and expand it if closed

Within GitLab.com

One other place to check for customer technical details is the
Account Management group on
GitLab.com. Just search by customer name and in the parent group and you should
find the Customer Collaboration Project. Most, but not all, premium and ultimate customers
should have one present.

Please note that these projects are most likely shared with the end customers as
well so updates on these projects are visible to the end-users, unlike the
organization notes within Zendesk
mentioned above.

SECTION: support/workflows/lost_emails.md

Overview

This workflow covers cases when a user requests that their email address be changed, for example due to having lost access to all email addresses on their account and being asked to perform Account email verification in order to access their account.

Stage 0: Ticket Triage

Ensure the ticket has the correct:

- Form SaaS Account
- Category, such as Cannot access account
- Subcategory, such as Need to change my username/email
- Impacted email address

Stage 1: Process

As the user has reportedly lost access to the email address associated with their GitLab.com account, they have likely raised the ticket using an alternate email address. As with all account activities, you should be particularly mindful of this and take care to not share any information related to the account which is not publicly available, or where applicable, account verification has not been successfully completed.

The actions support can take on accounts are different for free users and paid users. To confirm the user's tier status, search for the user using the User Lookup GitLab Super App in Zendesk to confirm the user's group memberships, if the user is not a member of any premium group they are considered a free user.

Paid user

Refer to Making Changes and Taking Actions on a user's behalf for the options available to paid users.

Free user

We are unable to take any action for free users who have lost access to all email addresses on their GitLab.com account. Apply the Zendesk macro Support::SaaS::GitLab.com::Email::Free user verification code and submit the ticket as Solved. Note that the support ticket will be closed after applying the macro, removing any opportunity for further response from the user, do not use the macro if you believe further dialogue is needed.

SECTION: support/workflows/malicious_ticket.md

Overview

Customers are inherently trusted partners and tickets are opened with positive intent. Although this will be true for the vast majority of tickets, it can happen that a resourceful adversary might purchase a GitLab license in order to try to abuse this trust.

Potential risks

Malicious tickets could be attempts to:

- Extract sensitive information about GitLab's infrastructure or systems
- Socially engineer support staff to bypass security controls
- Request configuration changes that would create security vulnerabilities
- Upload malicious files disguised as evidence or troubleshooting artifacts
- Establish rapport for future more sophisticated attacks
- Obtain escalated access or permissions beyond what is necessary

Warning signs

Although the following indicators will match legitimate behaviours of some clients, Support engineers should be alert to these potential indicators of malicious intent, especially when multiple of them are combined:

Unusual ticket content

- Requests that seem outside the normal scope of support
- Vague problem descriptions that require excessive back-and-forth
- Inconsistencies in the reported technical details
- Requests for sensitive information not required to resolve the stated issue

Customer behavior

- Excessive urgency or pressure to resolve quickly

- Attempts to build personal rapport followed by requests for exceptions to policy
- Reluctance to provide necessary information to troubleshoot (unless client has strict privacy restrictions like US Government or air gapped clients)
- Requesting support for capabilities that seem unrelated to the customer's known use case

Technical red flags

- Attachments with unusual file extensions or double extensions (.txt.exe)
- Links to externally hosted materials instead of direct attachments
- Links to external sites that require authentication with GitLab credentials
- Requests to run scripts or executables on GitLab systems
- Requests to disable security features or monitoring
- Requests to log in to their self-managed GitLab instance
- Invitations to an external GitLab instance or group

Response procedure

Initial assessment

1. Trust your instincts - if something feels unusual, take it seriously
2. Do not click on suspicious links or download attachments that seem unusual
3. Do not share sensitive information, even if the request seems urgent
4. Document all concerning aspects of the ticket

When in doubt, contact security

If you suspect a ticket may have malicious intent:

1. Do not communicate your suspicions to the customer
2. Continue normal communication while seeking guidance
3. Contact the security team via Slack by using /security and provide:
   - The ticket number
   - Specific elements that raised your concern
   - Any actions you've taken so far

After reporting

- Continue normal communication until advised otherwise by the security team
- Follow instructions provided by security
- Do not take exceptional actions requested by the customer without security approval

Remember

Most tickets are fortunately legitimate, this guidance is meant to help identify rare exceptions where caution is warranted. When in doubt, it's always better to consult with security than to ignore potential warning signs.

SECTION: support/workflows/marking_tickets_as_spam.md

Marking tickets as spam in Zendesk

Sometimes it can be necessary to mark a ticket as spam in Zendesk.
This suspends the end-user, and they won't be able to
submit tickets or access our Service Desk anymore.

There are two methods to mark a ticket as spam, depending on your permissions:

1. For Zendesk administrators and Support agents with Support Staff - CMOC role:
   - Click the dropdown menu on the right side of the ticket
   - Select Mark as spam

1. For all other Support agents:
   - Apply the Spam macro
   - This will solve the ticket without sending a CSAT survey.

SECTION: support/workflows/mattermost.md

This workflow is currently under review and can not be used in its current form. See https://gitlab.com/gitlab-org/omnibus-gitlab/-/issues/8594

Escalating to the Mattermost team

Mattermost has created a mattermost-support account in GitLab for support issues, and has subscribed to the mattermost label in the following projects:

- omnibus-gitlab
- gitlab-ce
- gitlab-ee

When a GitLab EE customer hits a Mattermost issue and you cannot reasonably resolve the issue using existing documentation:

- Do your best effort to make sure there is enough information to reproduce the issue
- Submit the issue in one of the mentioned projects and apply mattermost label. When the label is applied, an email notification is sent
to the technical support team who answers the question within two business days using the mattermost-support account.
- For Priority support (Premium/Ultimate customers, additionally assign the issue to the mattermost-support account. This assignment sends an email notification,
which is automatically escalated to the critical level technical support who answers the question within 4 hours using the mattermost-support account.

This information is taken from Service-Level Agreement (SLA)
page of Mattermost docs.

NOTE: Note:
The Mattermost team sometimes uses their personal accounts to respond to issues.

jasonblais is one such account.

Other resources

- Mattermost forum - has over a thousand people registered on the forum and every new question and answer makes thing easier to troubleshoot.

SECTION: support/workflows/meeting-frt-sla.md

The previous contents of this page have been moved to other pages within the Support Handbook, such as:

- Working on Tickets
- Support Engineer Responsibilities

SECTION: support/workflows/namesquatting_policy.md

Overview

According to the statement of support, namespaces may be released when they meet the appropriate criteria, and requested by a paid customer (member of a paid namespace or a self-managed customer migrating to SaaS).

IMPORTANT NOTE: If you have any situation that is unusual, or does not fall under the workflow below, open an Issue with Security Operations. Describe the situation and request them to review and provide guidance.

NOTE: When applying any of the macros ensure to replace the placeholder "REQUESTEDNAME" with the namespace requested.

Workflow

1. Search for the requested namespace in GitLab.com admin: users or groups, once found visit the GitLab admin page for the namespace.
1. Apply the Support::SaaS::Gitlab.com::Name Squatting Policy::Internal Checklist macro in Zendesk. Please remember, impersonating a user will reset the Last Sign-In values on the user account such as Last Sign-In IP and Last Sign-in at (Impersonation should be avoided when reviewing activity on a Personal Namespace).
1. Answer all questions in the Internal Checklist (Yes/No) ensuring to cross-check the information found in the admin section.
1. If the namespace is eligible for immediate release, follow Request successful.
1. If the namespace is eligible for release, but requires attempting to contact the owner, follow Namespace needs owner contact.
1. If the namespace is not eligible for release, follow: Namespace is not available.

Namespace needs owner contact

Contact Owner:

1. Create a new Zendesk ticket with the namespace owner's email address as the requester (found in admin) by following this specific workflow to create ticket and user
1. Apply the macro General::Outbound Contact Request that ensure the new ticket routes properly and the end-user we wish to contact receives the correct notification.
1. Apply the Support::SaaS::Gitlab.com::Name Squatting Policy::Contact Namespace Owner macro and mark the ticket as On-hold.
1. Make an internal comment providing a link to the namespace requester's ticket.

If the group contains multiples owners, contact one owner per ticket. Limit to 3 owners if more (you can pick the owners that have the most recent Last activity in the page https://gitlab.com/groups/<groupname>/-/groupmembers or/and the owner(s) that is(are) listed as Source if still an owner at the time of the namesquatting request).

Requester's Ticket:

1. Copy the ticket's link and add it to the Internal Checklist.
1. Reply to the requester with the Support::SaaS::Gitlab.com::Name Squatting Policy::First Response macro and mark ticket as On-hold.

Namespace owner responded

If the namespace owner makes a response (don't remove my namespace) follow these steps:

1. Use the following snippet as the response in the namespace owner's ticket and set the ticket to solved:

```
<details>
  <summary markdown="span">Namespace Owner Response - Received</summary>

  <p>Hi,</p>

  <p>Thank you for confirming that you wish to maintain control of the requested namespace. Per our Name Squatting Policy, we have cancelled this request and will not release your namespace.</p>

  <p>I'll mark this ticket as solved, please reach out if you have any further questions.</p>
</details>
```

1. Apply the Support::SaaS::Gitlab.com::Name Squatting Policy::Failed Namespace Request to the namespace requester's ticket.

Namespace owner has not responded

If after one week there has been no response, apply the Support::SaaS::Gitlab.com::Name Squatting Policy::Contact Namespace Owner macro a second time (replace within 2 weeks in the macro with the due date. Eg. before the 25th of March) and mark the ticket as On-hold.

After two weeks, the ticket will be automatically marked as open and an email sent to the

assigned engineer.

If the namespace owner has made no response, follow the Request successful steps.

Request successful

If the request is successful, follow these steps:

For users, change the owner's username with Chatops:

1. In Slack, run /chatops run user idle <ownerusername>.
1. Add an Admin note.

If you'd prefer to use admin, or for groups, rename the owner's namespace with these steps:

1. Navigate to the namespace in admin - users or groups
1. Select "Edit" on the profile.
1. Append "idle" to the username in case of a user, or to the group URL in case of a group.
1. Add an Admin note.
1. Save changes.

In Zendesk:

1. Apply the Support::SaaS::Gitlab.com::Name Squatting Policy::Successful Namespace Request macro to the Namespace requesters ticket and mark the ticket as Solved.

Namespace is not available

1. Apply Support::SaaS::Gitlab.com::Name Squatting Policy::Failed Namespace Request macro and mark ticket as Solved.

FAQs

1. Does a login in response to a name squatting request mean that the account is active?

No, the user has to explicitly reply to the name squatting request saying "I want to keep my namespace". If the user hasn't responded and has just logged in, send a final message saying something like, "I see you logged in at X, but you need to let us know here if you want to keep your namespace".

1. What constitutes data in the account?

A group, a project, etc. means data. Unless the project or group is empty, or there's been no activity for 2 or more years.

1. Is namespace squatting permitted?

Namespace squatting is not permitted as explicitly stated in our terms. User and group names are provided on a first-come, first-served basis and may not be reserved.

1. Does using a trademark in a way that has nothing to do with the product or service for which the trademark was granted considered a violation of trademark policy?

Using another's trademark in a way that has nothing to do with the product or service for which the trademark was granted is not a violation of trademark policy. Claiming trademark infringement is a legal process, and we will not release a namespace for trademark violation without a court order.

1. Should we release a namespace even if the request might seem questionable?

Yes, we should always release the namespace as long as it meets the release criteria. Trust and Safety will take the necessary steps to mitigate any abusive activity which may subsequently originate from the namespace. See Support Team Meta issue 3145 for discussion and additional context from Support, Legal, and Trust and Safety.

Macros

- Support::SaaS::Gitlab.com::Name Squatting Policy::Failed Namespace Request
- Support::SaaS::Gitlab.com::Name Squatting Policy::Internal Checklist
- Support::SaaS::Gitlab.com::Name Squatting Policy::First Response
- Support::SaaS::Gitlab.com::Name Squatting Policy::Contact Namespace Owner
- Support::SaaS::Gitlab.com::Name Squatting Policy::Successful Namespace Request

Automations

- Status::Open::Reopen namesquatting On-hold tickets after 1 week

---
SECTION: support/workflows/ooo-ticket-management.md

Overview

These workflows discuss how support engineers can asynchronously manage and summarize on-going assigned tickets within Zendesk before they go on PTO.

Using the OOO Ticket Summary macro

As part of this workflow, the Support Engineer going on leave will leave notes on currently Open, Pending and On-Hold tickets with a macro. This macro will provide a summary of the ticket, add the Support Engineer to the ticket's CC list, and adds the ooosummary summary tag to the ticket. It is recommended to follow this workflow for all high-priority tickets or when taking three or more days of PTO.

Ticket Prioritization Workflow

Before Going on PTO

- Optional: You may choose to document your tickets by making a list or taking a screenshot, including the description and ID numbers. This can serve as a helpful backup reference.
- Understand Automation: A ticket will automatically be unassigned and placed onto the Global Queue if: The tag ooosummary is applied to it AND the customer responds to the ticket.

For Severity 2 and Above Tickets

When planning PTO with high-priority tickets in your queue:

1. Find an appropriate assignee through the following progression:
   - Check for colleagues on the same shift.
   - Check regional support channels if no one is available on your shift.
   - If no assignee can be found, escalate to your manager.
1. Conduct a warm handover with the new assignee:
   - Schedule a pairing session or have a detailed Slack discussion.
   - Walk through the ticket details, customer context, and current status.
   - Apply the General::OOO Ticket Summary macro using the below Flow Chart Workflow.
   - The new Assignee takes assignment of the ticket.

For Severity 3 and Below Tickets

For lower-priority tickets:

1. Inform customers of your upcoming absence.
2. Set your ticket to Pending status.
3. Apply the General::OOO Ticket Summary macro to all tickets.

Workflow

Go to the My Assigned Tickets view in Zendesk. For each ticket you wish to summarize because you anticipate on-going work will be required, do the following:

1. Use the General::OOO Ticket Summary macro.
2. Fill in the sections of the internal note with details for your peers. It is important that you summarize:
   - What is the problem to be solved?
   - Action Taken?
   - Next Steps Needed? Alternatively, clarify if you are uncertain what the next steps are.
   - Blockers?
   - Return Date.
3. Feel free to also ask regional peers if they can pickup tickets in other forms of communication, such as Slack, but Zendesk should remain as the single source of truth for tickets that need attention from other team members.
4. At the end of your last work day before taking PTO, update your availability using the Out of Office app in Zendesk.
    1. Navigate to the app in Zendesk.
    1. If empty, select "Refresh the app" at the top of the page.
    1. Click the Make unavailable button in the row with your agent information. It is important that you do this for tickets to be unssigned when the customer responds.

Ticket Handover Process

When taking over a ticket that has the ooosummary tag:

1. Review unassigned tickets for your region from the Global Suppport Ticket View.
1. Remove the ooosummary tag from the ticket.
1. Set the Zendesk field Handover Status to Handover Completed.
1. Update ticket status and add appropriate comments for any work performed.
1. After the return date specified in the macro, you can liase with the original engineer to hand the ticket back. If needed, schedule a knowledge transfer session with the returning engineer.

Important: If you skip removing the tag ooosummary then the ticket will be automatically unassigned if the customer responds again.

Returning from PTO Process

1. Make yourself available in Zendesk by selecting "Make available" in the Out of office app.
1. Optional: Review the status of the tickets assigned to you before your PTO. You can coordinate with the new owner if it makes sense to reclaim ownership. This could be beneficial if you have an established rapport with the customer, possess strong technical expertise on the issue, or had previously agreed to continue the investigation upon your return.

PTO FlowChart

```
mermaid
flowchart TD
    Start[Engineer Planning PTO] --> DocumentTickets[Document Ticket List/Screenshot]
    DocumentTickets --> TicketPriority{Check Priority?}

    TicketPriority -->|Sev 2 and Above| HighPriority[High Priority Process]
    TicketPriority -->|Sev 3 and Below| NormalPriority[Normal Priority Process]

    HighPriority --> NotifyManager[Notify Manager 2+ Days Before]
    NotifyManager --> FindAssignee[Find Assignee]

    FindAssignee --> SameShift[Check Same Shift]
    SameShift --> OtherRegions[Check Regional Channels]
    OtherRegions -->|Found| WarmHandover[Conduct Warm Handover]
    OtherRegions -->|Not Found| EscalateManager[Escalate to Manager]

    WarmHandover --> PairingSession[Pairing Session/Slack Discussion]
    PairingSession --> ApplyMacro[Apply OOO Ticket Summary Macro]

    NormalPriority --> InformCustomer[Inform Customers of Absence]
    InformCustomer --> SetPending[Set to Pending Status]
    SetPending --> ApplyMacro[Apply OOO Ticket Summary Macro]


    subgraph MacroDetails [OOO Ticket Summary]
```

```
    ApplyMacro --> Summary[Add Ticket Summary]
    Summary --> AddOOOTag[Add ooosummary Tag]
    AddOOOTag --> AutoUnassign[Note: Ticket will auto-unassign if customer responds]
    AutoUnassign --> DocumentDetails[Document:
    - Problem Description
    - Action Taken
    - Next Steps Needed
    - Blockers
    - Return Date]
  end
```

SECTION: support/workflows/pairify.md

Overview

This document provides information on what Pairify is and how to use it to record pairing sessions.

What is Pairify?

Pairify is a Slack bot application that will scan monitored Slack channels on a schedule for any conversations reacted with (:pairify:).

It will then extract all Zendesk URLs, GitLab URLs (epics, issues, merge requests, handbook and docs), Slack participants/mentions and automatically create a pairing issue with the extracted URLs, participants (converted to GitLab.com usernames) and closes out the issue.

Using Pairify

NOTE: For Pairify to correctly map your Slack user to your GitLab.com user, you must configure your Slack profile's GitLab.com profile field. See edit your profile for more details.

To incorporate Pairify into your pairing session workflow:

1. Create a thread in one of the supported channels and pair with your colleagues!

1. Ensure that the thread conversation contains the following information:

   a. All Zendesk ticket URLs worked on

   b. All participants in the pairing session need to comment on the thread, or mentioned @slackusername in the thread

1. Optional. Specify the pairing session type.

1. React to the thread with the (:pairify:) emoji once the pairing is completed to mark a

conversation for Pairify to process.

You then need to wait for the next scheduled execution of Pairify, as explained in how Pairify works.

How Pairify works

Pairify executes every 30 minutes via a scheduled pipeline. When the pipeline begins, Pairify will search all monitored channels
for any conversation reacted with the (:pairify:) emoji that was created within the last 12 hours.

Pairify will then:

1. Extract any Zendesk URLs, participants and mentions within the conversation
1. Convert all Slack users to their GitLab.com counterparts
1. Create a GitLab issue in the Support Pairing project with the details extracted. The pairify label will be automatically applied to the issue, along with any other labels determined during processing.
1. Creates a message on the conversation thread indicating that the conversation has been processed. The response will also include a link to the issue created
1. React to the conversation with an (:issue-created:) emoji so the conversation is not processed again on subsequent executions.

Specifying the pairing session type

Pairify can apply additional labels to pairing issues to indicate the pairing session type if you react with additional emojis.

The reactions in this table are mutually exclusive - Pairify will only process the first reaction added to the conversation if multiple reactions are found:

| Emoji
| Shortcode | Description |
|---
---|---|---|
| | :crush: | Adds the crush label to the pairing issue
|
| | :customercall: | Adds the customer-call label to the pairing issue |
| | :office-hours: | Adds the office hours label to the pairing issue |

To ensure these reactions are picked up by Pairify, you should add these reactions before adding the :pairify: emoji to your conversation.

Channels monitored by Pairify

Refer to the .settings.production.yml file for a list of channels being monitored by Pairify.

Troubleshooting

- Refer to the troubleshooting section in the Pairify project.
- Review the issue tracker as you might be encountering a known issue.

SECTION: support/workflows/patching_an_instance.md

Patching an Omnibus install

Sometimes, we need to ask a customer to patch their systems manually. This may be because:

1. The customer can't upgrade to the newest version, but needs a fix from that version.
1. We have a fix merged, but not yet in a release.
1. The fix for their issue is still in development, but we'd like to verify that it resolves the customer's problem.

For Omnibus installs on a single server, this is fairly straightforward. Replace $mriid below with the IID of the merge request, or change the URL to point to a raw snippet.

shell
Ensure that the "patch" package is installed
Use the package manager specific to your Linux OS
For Ubuntu, for example, sudo apt install patch

curl -o /tmp/$mriid.patch https://gitlab.com/gitlab-org/gitlab/-/mergerequests/$mriid.patch
cd /opt/gitlab/embedded/service/gitlab-rails
patch -p2 -b -f < /tmp/$mriid.patch
gitlab-ctl restart

To revert the patch, use the .orig files the patch program generates.

A patch can include changes to multiple files. If you want to confirm that the file or files were patched,
you can compare the file(s) to the checksums included with most package managers.

For example, in Debian/Ubuntu running dpkg you can compare the checksum of each of the files in the patch to the list in /var/lib/dpkg/info/gitlab-ee.md5sums.
View the patch file in a text editor to see which files are affected.

shell
grep "opt/gitlab/embedded/service/gitlab-rails/<pathtofile>" /var/lib/dpkg/info/gitlab-ee.md5sums   The path does not have a leading /
md5sum /opt/gitlab/embedded/service/gitlab-rails/<pathtofile>

The checksums will differ if the patch was applied correctly.

Note:

- This process only applies to the Rails application (the GitLab repository).
Other components may need additional steps.
- The patch will need to be reapplied if GitLab is upgraded.
- The value of the patch option -p needs to be set to the correct a strip size.

Patching a Docker install

The GitLab Docker image uses an Ubuntu-based image with Omnibus installed inside the container to run GitLab. You can follow the same
steps as Patching an Omnibus install. Make sure to install patch binary from Ubuntu repository:

shell
docker exec -it <gitlab-container> bash
apt update && apt install patch -y
curl -o /tmp/$mriid.patch https://gitlab.com/gitlab-org/gitlab/-/mergerequests/$mriid.patch
cd /opt/gitlab/embedded/service/gitlab-rails
patch -p1 -b -f < /tmp/$mriid.patch
gitlab-ctl restart

Note:

- Deleting and recreating the container will revert the patch.
- The value for the patch option -p needs to be set to the correct a strip size.

Patching a Kubernetes install

Patching a Kubernetes install involves doing the following steps:

1. Identify the image we want to patch.

shell
Identify the image used for gitlab-webservice
kubectl -n <gitlab-namespace> get deployment <webservice-deployment> -o yaml | grep image:
image: registry.gitlab.com/gitlab-org/build/cng/gitlab-webservice-ee:v15.5.1
image: registry.gitlab.com/gitlab-org/build/cng/gitlab-workhorse-ee:v15.5.1
...

The command output will show a list of images, one of which you will need to patch. In this example, we would need to patch registry.gitlab.com/gitlab-org/build/cng/gitlab-webservice-ee:v15.5.1

1. Create a Dockerfile that we will use to build the image for the patch

txt
FROM registry.gitlab.com/gitlab-org/build/cng/gitlab-webservice-ee:v15.5.1

```
ARG MRIID

USER root

RUN apt-get update -y && apt-get install -y patch

USER git

RUN curl -o /tmp/$MRIID.patch https://gitlab.com/gitlab-org/gitlab/-/mergerequests/$MRIID.patch
RUN bash -c  "cd /srv/gitlab; patch -p1 < /tmp/$MRIID.patch"
```

Replace registry.gitlab.com/gitlab-org/build/cng/gitlab-webservice-ee:v15.5.1 with the image you identified in step 1.

1. Build and push the image with docker build and docker push:

```
shell
 Replace <mergerequestid> with the ID of the merge request containing the patch.
docker build --build-arg MRIID=<mergerequestid> -t path/to/remote/registry/gitlab-webservice-ee:v15.5.1 .
docker push path/to/remote/registry/gitlab-webservice-ee:v15.5.1
```

1. Update the deployment to use the patched image:

```
shell
 Re
```